

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON

CLASE 08 • NIVEL 1 - PRINCIPIANTE

Clase 08: Proyecto Integrador: Sistema CLI Completo

«Construyendo tu Primera Aplicación Real de Consola»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 08** del **Curso 1: Fundamentos Básicos de Python**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Integrar los 4 pilares de la programación en una arquitectura modular de software.

Competencia Práctica: Construir un gestor de tareas pendientes (Task Manager) con menú interactivo.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 08: Proyecto Integrador: Sistema CLI Completo

El proyecto integrador une todos los conocimientos adquiridos en el Curso 1 para crear una herramienta real.

☀ Metáfora Central: Construyendo tu Primera Aplicación Real de Consola

Construir tu primera aplicación es como armar tu propia bicicleta: cada pieza encaja para ponerla en marcha.

Principios Teóricos y Modelo Mental

Arquitectura modular: Separación de la interfaz de consola de la lógica de negocio.

Manejo de excepciones: Asegurar que entradas inválidas no detengan el programa.

⚡ Regla de Oro en Python

Estructura siempre tu punto de entrada con el patrón estándar `if __name__ == '__main__':`.

Arquitectura y Movimiento de Datos en Memoria

Arquitectura del bucle principal de eventos del sistema CLI.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Inicialización del estado en memoria.	Colección de tareas activa.
2. Evaluación	Despliegue del menú y captura de opción.	Espera de input en stdin.
3. Transformación	Despacho a la función correspondiente.	Mutación controlada.
4. Retorno / Salida	Confirmación visual y repetición del ciclo.	Bucle activo hasta salir.



Visualización Mental

Cada opción del menú debe llamar a una función especializada e independiente.

Código de Demostración en Python 3.10+

Estructura base del gestor de tareas en consola:

main.py (Python 3.10+)

PEP 8 Compliant

```
class TaskManager:
    def __init__(self):
        self.tasks = []

    def add_task(self, title: str):
        self.tasks.append({"id": len(self.tasks) + 1, "title": title, "done": False})

    def list_tasks(self):
        return self.tasks

tm = TaskManager()
tm.add_task("Aprender Python con Wisrovi")
print("Tareas registradas:", tm.list_tasks())
```

Análisis de Ingeniería del Código

Uso de clases, métodos encapsulados, listas de diccionarios y formateo de salida.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Buenas prácticas para proyectos integradores y código mantenible.

⚠ Gotcha Frecuente (Trampa de Principiante)

Escribir todo el código en un solo archivo plano sin funciones ni modularidad.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

500 líneas de código plano desordenado ❌

✅ Patrón Correcto

Funciones modulares y clases con responsabilidades únicas ✅

🛡 Consejo de Resiliencia en Producción

Documenta tus scripts con un README claro explicando cómo ejecutar la aplicación.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 08: Proyecto Integrador: Sistema CLI Completo**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Construyendo tu Primera Aplicación Real de Consola
2. Regla de Oro	Estructura siempre tu punto de entrada con el patrón estándar <code>if __name__ == '__main__':</code> .
3. Gotcha Crítico	Escribir todo el código en un solo archivo plano sin funciones ni modularidad.
4. Buenas Prácticas	Documenta tus scripts con un README claro explicando cómo ejecutar la aplicación.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Amplía el TaskManager para permitir marcar tareas como completadas y eliminarlas por ID.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.